2017

TSSVRAPI 说明文档

技术规范

TSXU

本文档所有内容均属于深圳市天软科技开发有限公司的商业机密，深圳市天软科技开发有限公司亦保留上述文档中阐述的包括而不限于技术、方法、思想的所有权。此文档仅仅只许可因本文档所必需传播的范围内使用（仅限于内部），本文档的获得者不得将本文档散发给不相关的其他机构和个人。

深圳天软科技 | 深圳市新闻路 1 号中电信息大厦 301-5

1 更新日志

1.1 更新日志

更新日志	更新说明
2017-05-18	文档创建和发布

1.2 摘要

本文介绍了天软.NET 平台的 API 常用接口.对开发包的程序做了整理和注解。需要更详尽的 API 接口请查看 TSSVRAPI.h 文件，开发包中有简单详尽的范例可供参考。

本文档内容描述了

- 环境 TSSVRAPI 开发包的介绍和配置说明
- 数据类型介绍
- 与天软交互执行
- 相关范例

2 TSSVRAPI 介绍

TSSVRAPI 是天软科技 2016 年推出的软件接口 API，用户可通过 TSSVRAPI 开发应用软件，访问天软平台数据，调用模型，定制天软平台插件，实现用户系统和天软平台的整合

TSSVRAPI 接口，可用于 C、C++、C#、Java 等高级语言与 TS 的交互。

3 TSSVRAPI 开发包

1. 到天软首页下载安装天软客户端(http://www.tinysoft.com.cn/tsdn/helpdoc/index.tsl?type=10124)
2. 下载 API 开发包(请联系天软市场人员)
3. 开发包中包括：

   1）头文件 TSSVRAPI.h：API 的结构、宏和函数的声明和说明；

   2）TSSVRAPI.lib：是已编译好的 TSSVR 的库。

   天软提供了 32 位的 lib 文件（\TSSVRAPI\Examples\lib\tssvrapi.lib）和 64 位 lib 文件（\TSSVRAPI\Examples\lib\x64\tssvrapi.lib）。用户可以根据本机的情况选择需要的 lib 位数。

   另外还提供了一个 TSSVRAPI.def 文件。如果您使用的编译器和 TSSVRAPI.lib 不兼容，可以自己生成对应编译器的 lib 文件,生成方法参考

   http://www.cnblogs.com/dongzhiquan/archive/2009/08/04/1994764.html

   3）Example：包括 tslsamasbuf（数据存取例子）、tslsamlocal（本地调用例子）、tslsamplugin（插件例子）、tslsamremote（远程访问例子）、tslsamsubscription（行情订阅例子）等。用户可以根据自己的需要，查看相关范例的代码。
4. 正确配置工程。

   1）正确引用 lib 库文件。在配置属性下的链接器中，附加库目录根据创建的工程的位数，选择对应的 lib 文件目录。比如 64 位的，选择 tssvrapi 的 64 位 lib 所在目录 \TSSVRAPI\Examples\lib\x64

   2）将编译输出文件输出到天软的安装目录(例如 D:\Program Files (x86)\Tinysoft\Analyse.NET),在此目录下有其运行需求的环境。

   3）配置属性下的常规选项字符集，选择多字节字符集。

   4）正确引用 TSSVRAPI 头文件

4 TSL 数据类型说明

4.1 常用数据类型

数据类型	说明	值
TSL_TINT	32位整数	0
TSL_TNUMBER	64位浮点	1
TSL_TSZSTRING	字符串	2
TSL_TTABLE	数组	5
TSL_TSTRING	字符串下标,仅作为数组的下标	6
TSL_TGRAPH	图形	8
TSL_TGRAPHGROUP	图形组	9
TSL_TNIL	NIL类型	10
TSL_TBINARY	二进制类型	11
TSL_TUNKNOWN	COM类型IUNKNOWN	14
TSL_TDISPATCH	COM类型IDISPATCH	15
TSL_TANY	TSL面向对象类型	16
TSL_TMatrix	Matrix类型,MSelect返回该数据类型	17
TSL_TINT64	64位整数	20
TSL_TNONE	无类型，此类型不会被使用，仅作为特殊用途	-1

4.2 TSL_State*

这是解释器状态指针，是 TSL 语言内部存放解释器状态的数据结构。

TSL_State* TS_GetGlobalL()和 TSL_InterpNewLWrap()、TSL_InterpGetLFromWrap()获取解释器状态指针。其中，TS_GetGlobalL 获取的指针不需要释放，一个线程只能得到一个指针。而 TSL_InterpNewLWrap 和 TSL_InterpGetLFromWrapde 获取的指针需要用 TSL_InterpFreeLWrap()释放。一个线程能创建多个指针。但是无论哪种方式获取的解释器状态指针都不能跨线程使用。

4.3 简单数据类型存取

4.3.1 创建和释放

TSL 语言中，所有的数据类型都需要创建（使用前）和释放（使用后），如果不释放数据对象，有可能造成内存泄露。

1）创建数据对象函数：`TObject* TSL_NewObject();`

2）释放数据对象函数：`void TSL_FreeObj(TSL_State* L, TObject* obj);`

释放 TSL 对象的内容，并重置为 NIL 类型：void TSL_FreeObjectContent(TSL_State* L, TObject* obj);

3）复制 TSL 对象：void TSL_DupObject(TSL_State* L, TObject* dest, TObject* src, bool copy);

支持在不同的解释器实例间复制（例如多线程中，非线程安全，需用户自己保障）：

void TSL_DupObjectEx(TSL_State* L, TSL_State* LS, TObject* dest, TObject* src, bool copy);

其中，

L:目标解释器状态指针

LS:源解释器状态指针

dest:目标TSL对象

src:源TSL对象

copy:为真则复制内容，为假则快速复制，并将原值设置为内容为虚值。正常情况下应为真，但特殊情况下如源值使用后立即会被释放则可以加速。不正确的使用可能会导致内存混乱

范例 1： 创建一个整数对象

```
TObject* int_obj=TSL_NewObject();  //创建整数对象
TSL_SetInt(TSL_State* L,int_obj,1);   //赋值整数对象 int_obj 为 1
```

范例2：初始化一个数组

```
TObject* objt1 = TSL_NewObject();
TSL_SetType(objstate, objt1, TSL_TTABLE); //设置为数组
```

4.3.2 数据赋值

除了两个基本的对数据对象创建和释放的函数，天软还提供了各种对简单数据操作的函数。比如我们在创建完一个数据对象之后，需要对数据对象赋值，下列的这些，是对各类数据对象赋值的函数：

1）设置32位整数：

```
void TSL_SetInt(TSL_State* L, TObject* o, int i);
```

2）设置64位整数：

```
void TSL_SetInt64(TSL_State* L, TObject* o, __int64 i);
```

3）32位下设置为32位整数，64位下设置为64位整数：

```
void TSL_SetIntPtr(TSL_State* L, TObject* o, INT_PTR i);
```

4）设置浮点数：

```
void TSL_SetReal(TSL_State* L, TObject* o, double r);
```

5）设置字符串：

```
bool TSL_SetString(TSL_State* L, TObject* o, char* str);
```

6）设置特定长度的字符串，允许非0尾：

```
bool TSL_SetStringEx(TSL_State* L, TObject* o, char* str, int len);
```

7）设置字节流类型：

```
bool TSL_SetBin(TSL_State* L, TObject* o, char* str, int len);
```

8）字符串下标类型：

```
bool TSL_SetStringIndex(TSL_State* L, TObject* o, char* str);
```

9）设置字符串下标类型，允许非 0 尾：

```
bool TSL_SetStringIndexEx(TSL_State* L, TObject* o, char* str, int len);
```

10）设置 TSL 对象为指定类型，如果类型已经是指定类型则不变，否则释放原类型初始化成新类型：

```
void TSL_SetType(TSL_State* L, TObject* p0, int T);
```

4.3.3 数据类型判断和读取

我们在对某个数据对象赋值的时候，一般都是知道该数据对象是什么类型的。但是我们对某个返回的数据对象读取的时候，就不一定都知道这是个什么数据对象了。比如执行一句天软语句后，预期返回一个数组数据，但是有可能执行没成功或出现其他问题，导致返回一句报错，即字符串语句。如果没有对数据类型判断，很有可能导致后面的执行报错。

所以读取这个数据对象之前，我们必须先判断数据对象类型。下列几个函数，是 TSL 提供的常用的数据对象。

1）得到 TSL 对象的类型值：

```
int TSL_GetType(const TObject* p0);
```

2）检查是否为 NIL 类型：

```
bool TSL_NilCheck(const TObject* p0);
```

3）检查类型为 32 位整数：

```
bool TSL_IntCheck(const TObject* p0);
```

4）检查类型为 64 位整数：

```
bool TSL_Int64Check(const TObject* p0);
```

5）检查类型为 64 位浮点

```
bool TSL_DoubleCheck(const TObject* p0);
```

6）检查是否是数字类型,也就是 TSL_TINT，TSL_TINT64，TSL_TNUMBER 之一

```
bool TSL_NumberCheck(const TObject* p0);
```

7）是否是字符串类型

```
bool TSL_StringCheck(const TObject* p0);
```

8）是否数组为非空数组

```
bool TSL_HashAsBool(Hash *t);
```

9）是否为数组类型

```
bool TSL_HashCheck(const TObject* p0);
```

10）是否为 Matrix 类型

```
bool TSL_MatrixCheck(constTObject* p0);
```

11）是否是字符串下标类型

```
bool TSL_StringIndexCheck(const TObject* p0);
```

12）是否是二进制类型

```
bool TSL_BinCheck(const TObject* p0);
```

13）是否可以当成字符串类型，也就是字符串、字符串下标、二进制类型其中之一

```
bool TSL_StringCheck2(constTObject* p0);
```

判断数据类型之后，可以用该数据类型对应的函数对数据进行读取。读取数据的函数，天软提供了如下的函数，用户可以根据需要进行调用：

1）将 TSL 对象当成整数，如果 TSL 对象本身类型为浮点、64 位整数则会取整为 32 位整数（转换可能溢出），否则结果为 0

```
int TSL_AsInt(constTObject* p0);
```

2）将TSL对象当成64位整数，如果TSL对象本身类型为浮点、32位整数则会取整为64位整数（转换可能溢出），否则结果为0

```
__int64 TSL_AsInt64(const TObject* p0);
```

3）32位TSL下相当于TSL_AsInt,64位下相当于TSL_AsInt64

```
INT_PTR TSL_AsIntPtr(const TObject* p0);
```

4）将TSL对象当成64位浮点，如果TSL对象本身类型为32位或64位整数则会转为浮点数（64位整数转换可能会丢失精度），否则结果为0

```
double TSL_AsDouble(const TObject* p0);
```

5）是否为真，空字符串，整数0，INT64 0，浮点0，NIL，二进制数组为空，长度为0的矩阵，空数组，TSL_TNONE类型返回假，否则为真

```
bool TSL_AsBool(const TObject*p0);
```

6）将 TSL 数据类型当成字符串访问

```
constchar* TSL_AsString(const TObject* p0);
```

7）获得字符串长度

```
int TSL_StringLen(const TObject* p0);
```

8）取出 TSL 对象的数组对象

```
Hash* TSL_GetHash(const TObject* p0);
```

9）获取数组的下标值

```
void TSL_HashGetIndexes(TSL_State* L, Hash*t, TObject** &Keys, size_t& len, bool needAll);
void TSL_GetIndexes(TSL_State* L, constTObject* obj, TObject** &Keys, size_t& len, bool needAll);
```

其中，

```
L:解释器状态指针
t:数组对象
obj:TSL对象，应为数组类型
Keys:返回下标缓冲区，使用完后TSL_Free(Keys)
len:下标个数
needAll:内部使用，设置为true
```

范例：

```
std::string s = "return rand(array('a',3,5,'b','c'),5);";//天软语句串
LWrap* lwrap = TSL_InterpNewLWrap();//创建解释器环境，可以一个线程创建多个
TSL_State* L = TSL_InterpGetLFromWrap(lwrap);//从解释器环境中获取 L
TObject* v = TSL_NewObject(); //返回值
if (TSL_ScriptGo(L, s.c_str(), v) == TSL_OK) //执行语句
{
    if (TSL_GetType(v) == TSL_TTABLE)//如果为数组结构
```

```
    {
        TObject** Key;
        size_t len;
        cout <<"get index by object\r\n";
        TSL_GetIndexes(L, v, Key, len, false);//获得所有下标
        for (int i = 0; i <len;++i)
            cout << TSL_ObjToStr(L,Key[i])<<"\r\n"; //打印每个下标
        TSL_Free(Key);//释放空间
        cout <<"get index by hash\r\n";
        TSL_HashGetIndexes(L,TSL_GetHash(v),Key,len,false);//获得所有下标
        for (int i = 0; i < len; ++i)
            cout << TSL_ObjToStr(L, Key[i]) <<"\r\n"; //打印每个下标
        TSL_Free(Key);//释放空间
    }

}
else
{
    printf("execute err:\r\n");
}
TSL_FreeObj(L, v);//释放返回的 tsl 对象
TSL_InterpFreeLWrap(lwrap);//释放解析环境
```

4.3.4 数组支持函数

由于数组的操作比较复杂，天软也提供了一系列数组操作的支持函数。

4.3.4.1 数组创建和复制

1）新建一个数组对象

```
Hash *TSL_NewHash(TSL_State *L, int nhash);
```

2）设置对象对数组类型，并且预先分配数组的存贮空间（仅仅是存贮空间，没有预先设定下标），预先分配在大数组时效率会高

```
void TSL_ForceTable(TSL_State* L, TObject* obj, int size);
```

3）仅释放 TSL_NewHash 的数组对象，TSL_GetHash 得到的对象无需释放

```
void TSL_FreeHash(TSL_State *L, Hash *t);//
```

4）复制数组

```
void TSL_DupHash(TSL_State* L, Hash* dest, Hash* src);//
```

5）在多个解释器状态指针间复制数组，与 TSL_DupObjectEx 类型

```
void TSL_DupHashEx(TSL_State* L, TSL_State* LS, Hash* dest, Hash* src);//
```

6）返回数组中的最大整数下标值+1

```
int TSL_HashGetN(Hash *t); //
```

7）返回数组中的下标数量

```
int TSL_HashRawGetN(Hash *t); //
```

8）取出数组中整数下标的最大值和最小值,返回数组的下标数量

```
int TSL_HashGetBounds(Hash *t, int* low, int* high); //
```

9)取出数组中值类型为 T 的的整数下标的最大值和最小值,并返回符合的下标数量,当 T 为 TSL_TNONE 与 TSL_HashGetBounds 行为相同

```
int TSL_HashGetIntIndex(Hash* t, int&Low, int& High, int T = TSL_TNONE);//
```

10）返回数组中的字符串下标数量

```
int  TSL_HashGetStringIndexCount(Hash* t);//
```

11）获得整数下标个数和字符串下标个数

```
bool TSL_HashGetItemSize(TSL_State* L, TObject* p0, int* iSize, int* sSize);//
```

12）获得最大可能需要的存放的整数下标的长度，并得到字符串下标的个数

```
bool TSL_HashGetItemMaxSize(TSL_State* L, TObject* p0, int* iMaxSize, int* sSize);
```

其中，

L:解释器状态指针

p0:TSL对象

iMaxSize:返回整数下标的最大个数，用来分配TSL_HashGetSubItems中的iIndexes的空间大小

sSize:字符串下标的个数

返回值：如果非数组类型，返回为假

13）获得下标值

```
bool TSL_HashGetSubItems(TSL_State* L, TObject* p0, int* iIndexes, constchar** sIndexes, int* sIndexLens, int* iSize, int* sSize);
```

其中，

L:解释器状态指针

p0:TSL对象

iIndexes:用于返回整数下标的缓冲区，空间应分配足够大，预先使用TSL_HashGetItemSize获得

sIndexes:用于返回字符串下标的缓冲区，空间应分配足够大，预先使用TSL_HashGetItemSize获得

sIndexLens:用于返回字符串下标值长度的缓冲区，空间应足够，预先使用TSL_HashGetItemSize获得

iSize:返回整数下标的个数。

sSize:返回字符串下标的个数

返回值：如果非数组类型，返回为假

4.3.4.2 数组取值赋值

需要使用 hash 值

1）用下标 Key 获得数组的 Value

```
constTObject *TSL_HashGet(TSL_State *L, constHash *t, constTObject *key);
```

2）取整数下标的值

```
constTObject *TSL_HashGetInt(constHash *t, int key);
```

3）取字符串下标的值

```
constTObject *TSL_HashGetSZString(TSL_State* L, constHash *t, constchar *key);
```

4）取字符串下标的值(带字符串长度)

```
constTObject *TSL_HashGetSZStringEx(TSL_State* L, constHash *t, constchar *key, int len);
```

不需要取出 Hash 值，而直接用 TSL 对象取下标

1）用下标 Key 获得数组的 Value

```
constTObject* TSL_HashGetItem(TSL_State* L, constTObject* p0, constTObject* Index);
```

2）取整数下标的值

```
constTObject* TSL_HashGetItemInt(TSL_State* L, constTObject* p0, int index);
```

3）取字符串下标的值

```
constTObject* TSL_HashGetItemSZString(TSL_State* L, constTObject* p0, constchar* sIndex);
```

4）取字符串下标的值(带字符串长度)

```
constTObject* TSL_HashGetItemSZStringEx(TSL_State* L, constTObject* p0, constchar* sIndex, int len);
```

设置数组获得数组的值引用（赋值）

1）用下标 Key 设置得到值引用

```
TObject *TSL_HashSet(TSL_State *L, Hash *t, TObject *key);
```

2）设置整数下标

```
TObject *TSL_HashSetInt(TSL_State *L, Hash *t, int key);
```

3）设置字符串下标

```
TObject *TSL_HashSetSZString(TSL_State *L, Hash *t, constchar *key);
```

4）设置字符串下标(带字符串长度)

```
TObject *TSL_HashSetSZStringEx(TSL_State *L, Hash *t, constchar *key, int len);
```

不需要取出 Hash 值，而直接用 TSL 对象进行设置（赋值）

1）用下标 Key 设置得到值引用

```
TObject* TSL_HashSetItem(TSL_State* L, TObject* p0, TObject* Index);
```

2）设置整数下标

```
TObject* TSL_HashSetItemInt(TSL_State* L, TObject* p0, int index);
```

3）设置字符串下标

```
TObject* TSL_HashSetItemSZString(TSL_State* L, TObject* p0, constchar* sIndex);
```

4）设置字符串下标(带字符串长度)

```
TObject* TSL_HashSetItemSZStringEx(TSL_State* L, TObject* p0, constchar* sIndex, int len);
```

删除或更名数组的下标

1）删除数组的某个下标

```
bool TSL_DeleteIndex(TSL_State* L, Hash* t, TObject* index, int* pos);
```

其中，

L:解释器状态指针
t:数组对象
index:数组下标
pos:存贮删除的位置，内部使用

返回：为真则删除成功

2）对行或列下标更名

```
bool TSL_Reindex(TSL_State* L, Hash* t, TObject* src, int cnt);
```

3）重构数组

```
bool TSL_Reshape(TSL_State* L, Hash* t, Hash* desHash, TObject* src, int cnt);
```

5 执行说明

第三方通过 TSSVRAPI 方式与 TS 交互有两种方式，在本地执行或是远程执行。本地执行即调用了本地 TSL 编译器的函数执行。远程执行即将与天软服务建立连接，将要执行的语句发送到天软服务中执行，执行出结果再传回第三方。

下面提供本地执行需要调用的函数，及相关的案例说明。

5.1 本地执行

本地执行需要使用 TSL_ScriptGo 函数执行 TSL 语句。TSL_ScriptGo 函数说明如下：

```
int TSL_ScriptGo(TSL_State* L, constchar* Content, TObject* v);
```

说明：本地执行TSL语句

参数：

L:解释器状态指针
Content:TSL语句
v:返回的结果集

返回值：TSL_OK成功，TSL_ERROR失败

范例：

```
std::string s = "a:=rand(5,5);b:=a+1;return b;";//天软语句串
LWrap* lwrap = TSL_InterpNewLWrap();//创建一个解释器环境，一个线程可创建多个
TSL_State* L = TSL_InterpGetLFromWrap(lwrap);//从解释器环境中获取 L
TObject* v = TSL_NewObject(); //返回值
if (TSL_ScriptGo(L, s.c_str(), v) == TSL_OK) //执行语句，TSL_OK 表示执行成功
{
    char* sV = TSL_ObjToStr(L, v); //将返回的 tsl 对象转换为 tsl 字符串
    printf("ans:\r\n\t%s\r\n", sV); //打印
    TSL_Free(sV); //释放字符串
}
```

```
    else
    {
        TSL_InterpErrorInfo* info = TSL_InterpNewErrorInfo(); //初始
        TSL_InterpGetLastError(info); //获得错误信息,包括行号和错误提示内容
        int line;
        constchar* errinfo = TSL_InterpGetErrorInfo(info, &line);//将错误行
号和提示内容分离
        printf("err:\r\nline:%d\r\ninfo:\r\n%s", line, errinfo);
        TSL_Free(info);//释放错误信息
    }
    TSL_FreeObj(L, v);//释放返回的 tsl 对象
    TSL_InterpFreeLWrap(lwrap);//释放解析环境
```

5.2 远程执行

5.2.1 登陆

远程执行，即必须使用账号登陆天软，包括三部分：

- 连接服务器
- 选择服务器类型
- 登陆账号

5.2.1.1 连接服务器

连接远程服务，需要提供连接的服务器的地址等信息，下面给出了连接的服务器数据结构，及相关的连接函数和范例。

```
//登录用的代理服务器结构
struct TProxyInfo{
    char ProxyAddress[256]; //代码服务器地址
    int proxyPort;//代理服务器端口
    char User[256];//代理服务器用户名
    char Pass[256];//代理服务器密码
    char Domain[256];//保留字段，代理服务器用户域信息
    int AuthMode;//保留字段，代理服务器认证模式
};
void TS_SetSocketInfo(constchar* Address, int Port, TProxyInfo* ProxyInfo);
//设置连接的 Socket 信息，如果不使用 ProxyInfo，则设置为 NULL
int TS_ConnectSocket();//创建连接，非0失败
void TS_DisconnectSocket();//断开连接
bool  TS_GetSocketConnected();//是否已经连接
```

范例：

```
/*
连接服务器使用范例
用于连接天软服务器，登陆，等操作，包含代理登录天软服务器的使用
*/
#include"stdafx.h"
#include"Windows.h"
#include<iostream>

extern"C" {
#include"../../TSSVRAPI.h"
};
#pragmapack( push, enter_lobject )

#pragmapack(4)

usingnamespace std;
#pragmapack( pop, enter_lobject )
int_tmain(intargc, _TCHAR* argv[])
{
    LWrap* lwrap = TSL_InterpNewLWrap();
    TSL_State* L = TSL_InterpGetLFromWrap(lwrap);
    TObject* v = TSL_NewObject();
    void * buf;

    //------>>>连接服务器参数设置，代理连接的使用
    TProxyInfo * proxyInfo = TSL_NewProxyInfo();
    /* 本代理参数为虚拟的，仅提供调用代理连接的语法范例 */
    char *addr = "192.168.0.1";int port = 10000;char *user = "user_one";char
*pass = "123456789";char * domain = "";int authmode = 0;
    TSL_SetProxyInfo(proxyInfo, addr, port, user, pass, domain, authmode);//
设置代理服务器信息块的内容
// 代理连接 TS_SetSocketInfo("tsl.tinysoft.com.cn", 443, proxyInfo);//设置连
接的 Socket 信息，如果不使用 ProxyInfo，则设置为 NULL
    TS_SetSocketInfo("tsl.tinysoft.com.cn", 443, NULL);//---设置连接参数，设
置 socket 连接的 ip,端口以及代理块的信息，此时为 NULL,不使用代理
    int res = TS_ConnectSocket();//---创建连接，非 0 失败
    if (res) {
        cout <<"connect fail "<< endl;
    }
    if(!TS_GetSocketConnected()) {//---检测是否已经连接,非 0 失败,判断当前 socket
是否连接上 tsl.tinysoft.com.cn:443.
```

```
        cout <<"socket connect fail "<< endl;
    }

    //----->>>登录服务器
    if(!TS_GetLogined()) {//---检测是否已登录,非 0 未登录
        cout <<"当前没有用户登陆 tsl.tinysoft.com.cn:443"<< endl;
    }
    charErrBuf[1024];
    int ErrBufLen = 0;
    if (TS_LoginServer("tswuxin", "wuxin", ErrBuf, ErrBufLen)) {//---登录服务器，非 0 失败
        cout <<"logginServer fail"<< endl;
    }
    //以别名模式登录服务器，非 0 失败,别名请参与相应的 TSL 函数手册
    //int TS_DefaultConnectAndLoginServer(TSL_State* L, const char* Alias, int* ErrNo, char* Error, int ErrorLen);

    //----->>>选择服务器，设置服务器登陆参数
     TS_SetSelectService("auto");//当有多个服务器建立连接时，选择一个连接的服务器，选择连接上的服务器的服务名,仅当存在多套服务的时候有意义。自动为"auto"
     cout <<"GetSelectService="<< TS_GetSelectService() << endl;;//获得当前使用的服务名
    //服务器有 32 位的版本和 64 位的版本，一旦两者同时存在的时候，可以设置计算位数来选择，未来主要支持 64 位版本
     TS_SetComputeBitsOption(2);//设置计算位数,0 自动，1：32 位，2:64 位
     cout<<"服务器计算位数="<<(int) TS_GetComputeBitsOption()<<endl;//获得所选择的服务器的计算位数

    //---->>>获取当前服务器地址端口
    cout <<"LoginedUser="<< TS_GetLoginedUser() << endl;; //获得已经登录的用户名，如果没登录则为 NULL
    cout <<"RemoteAddress ="<< TS_GetRemoteAddress() << endl;//获得远程地址
    cout <<"RemotePort="<< TS_GetRemotePort();//获得远程端口
    TS_DisconnectSocket();//断开连接
    cout <<"连接服务器测试，end ."<< endl;
    TSL_FreeObj(L, v);
    TSL_InterpFreeLWrap(lwrap);
    return 0;
}
```

5.2.1.2 选择服务器类型

服务器存在 32 位和 64 位版本的，一旦两者同时存在的时候，可以设置计算位数来选择，未来主要支

持 64 位版本。用户需要通过下列函数设置来选择需要的版本。

1）选择连接上的服务器的服务名,仅当存在多套服务的时候有意义。自动为"auto"

```
void TS_SetSelectService(constchar* s);
```

2）获得当前使用的服务名

```
constchar* TS_GetSelectService();
```

3）设置计算位数,0 自动，1：32 位，2:64 位

```
unsignedchar TS_SetComputeBitsOption(unsignedchar Option);
```

4）获得所选择的服务器的计算位数

```
unsignedchar TS_GetComputeBitsOption();
```

5.2.1.3 登陆账号

用户在远程执行时，需要给定用户账号和密码以及登录的服务器地址等数据，才能登录。登录的函数及范例见下。

1）登录服务器函数，非0表示失败

```
int TS_LoginServer(constchar* UserName, constchar* Password, char* ErrBuf, int ErrBufLen);
```

2）以别名模式登录服务器，非0表示失败,别名的配置请参考相应的TSL函数手册

```
int TS_DefaultConnectAndLoginServer(TSL_State* L, constchar* Alias, int* ErrNo, char* Error, int ErrorLen);
```

3）判断是否已登录

```
bool  TS_GetLogined();
```

范例：

```
TS_SetSocketInfo("tsl.tinysoft.com.cn", 443,NULL);
if (0 == TS_ConnectSocket())
{
    char buf[1024];//登陆信息
    if (0 != TS_LoginServer("用户名", "密码", buf, sizeof(buf)))
    {
        printf("Login error:%s\r\n", buf);
    }
    else
    {
        printf("Login success\r\n");
    }
}
else
{
    printf("Connect error!\r\n");
}
```

5.2.2 执行

连接服务器且登录成功之后，可以将天软语句发送到远程服务器中执行，获取返回结果。下面提供了远端执行的函数及其说明和范例：

```
void TS_SendExecuteAndWait(TSL_State* L, constchar* ExecStr, TObject* SysParam, TObject* oResult, TObject* ExecResult, TObject* ErrObj, bool Online);
```

说明：将天软语句发送到远程服务端执行。

参数：

L:解释器状态指针

ExecStr:需执行的TS语句

SysParam:初始的系统参数变量，应为TSL_TTABLE类型

oResult:返回值，为整数0成功

ExecResult:执行的结果

ErrObj:执行失败返回错误信息

Online:为真在线执行，为假委托执行

范例：

```
LWrap* lwrap = TSL_InterpNewLWrap();//构造解析环境
TSL_State* L = TSL_InterpGetLFromWrap(lwrap);//获得解析环境的状态指针
TObject* r = TSL_NewObject(); //执行是否成功
TObject* v = TSL_NewObject(); //执行返回值,tsl 语句的 return 结果
TObject* err = TSL_NewObject(); //执行错误信息
if (TS_GetLogined())//如果已经登陆
{
    TS_SendExecuteAndWait(L, "return select datetimetostr(['date']) as 'time',['StockID'],['price'] from tradetable datekey 20160114T to 20160114T+9.6/24 of array('SH000001') end;", (void*)TSL_GetGlobalNilObj(), r, v, err, true); //如果无需系统参数，直接使用TSL_GetGlobalNilObj，否则需要生成一个系统参数的对象
}
if (TSL_IntCheck(r) && TSL_AsInt(r) == 0)
{
    char* sV = TSL_ObjToStr(L, v);//将tsl对象转换为字符串
    printf("ans:\r\n\t%s\r\n", sV);
    TSL_Free(sV);
}
else
{
    printf("Error:\r\n");

}
//释放tsl对象和解析器
TSL_FreeObj(L, v);
```

```
TSL_FreeObj(L, r);
TSL_FreeObj(L, err);
```

5.3 异步处理和数据订阅

如果函数的执行需要长时间处理，TSSVRAPI 支持异步处理，执行过程中，可以有中间输出数据。异步处理的函数为 TS_SendExecuteAndWaitEx，该函数调用发出后，函数立即返回，进行后续处理。同时，改函数可设置两个回调函数参数 TExecuteASyncCallBack callback 和 TExecuteASyncFinal FinalCallBack。

在服务器处理调用请求时，调用体内可以通过 echo 或 sendrunning 将处理中的信息发送给调用段，调用端通过 callback 处理这些信息。在服务器处理完调用请求后，会回调 FinalCallBack 函数，告知处理完成。

下面是异步处理函数的说明和相关范例。

```
int TS_SendExecuteAndWaitEx(TSL_State* L, constchar* ExecStr, TObject* SysParam, TObject* oResult, TObject* ExecResult, TObject* ErrObj, bool Online, TExecuteASyncCallBack callback, TExecuteASyncFinal FinalCallBack);
```

说明：异步处理函数。支持远程数据处理的同时，将处理信息回调到调用端。

参数：

L:为解释器状态指针
ExecStr:需执行的TS语句
SysParam:初始的系统参数变量，应为TSL_TTABLE类型，应包括字段
Type：1
IDs：订阅证券代码的一维数组
SUBs：订阅数据项的名称
oResult:返回值，为整数0成功
ExecResult:执行的结果
ErrObj:执行失败返回错误信息
Online:为真在线执行，为假委托执行
callback:执行的回调函数,为NULL则为同步模式
FinalCallBack:异步执行的时候结束的回调函数

其中，TExecuteASyncCallBack类型定义：

```
typedefvoid(*TExecuteASyncCallBack) (TSL_State* L, int RecvType, int ChannelId, int ErrNo, char* ErrMsg, TObject* ResultObj, TObject* EnvObj);
```

说明：执行时的回调，主动 EndExecute C++接口不回调

参数：

L:为解释器状态指针
RecvType:接收的类型信息
0:错误通知
0x0201:执行函数返回
0x0301:提交委托函数的状态返回

0x0401:Echo

0x0402:RunningData返回

ChannelId:返回信息的句柄ID

ErrNo:错误号(Echo的时候为Echo的长度)

ErrMsg:错误信息(Echo的时候为Echo的内容,RunningData为标识串)

ResultObj:返回的结果数据(委托执行、Echo、错误通知此项为NULL)

EnvObj：保留参数

返回值：返回执行的句柄。可以用TS_EndExecute终止执行

TExecuteASyncFinal类型定义：

```
typedefvoid(*TExecuteASyncFinal) (TSL_State* L, int ChannelId);
```

说明：异步执行的时候结束的回调

参数：

L:为解释器状态指针

ChannelId:返回信息的句柄ID

```
void TS_EndExecute(int Channel);
```

说明：取消订阅

参数：

Channel：线程ID

范例：

```
void callback(TSL_State* L, intRecvType, intChannelId, intErrNo, char* ErrMsg,
TObject* ResultObj, TObject* EnvObj)
{
    printf("ChannelId:%d Type:%d\r\n", ChannelId, RecvType);
    switch (RecvType)
    {
    case 0: //执行错误返回
    case 0x0301://委托执行信息返回
        printf("ErrorNo:%d,Error:%s\r\n", ErrNo, ErrMsg);
        break;
    case 0x0201: //函数执行返回
        printf("Result return:\r\n");
        printf("data: %s\r\n", TSL_ObjToStr(L, ResultObj));
        printf("\r\n");
        break;
    case 0x0401:// echo 数据
    {
        char* s = (char*)TSL_Malloc(ErrNo + 1);
        memcpy(s, ErrMsg, ErrNo);
        s[ErrNo] = 0;
        printf("Echo:%s\r\n", s);
```

```
        TSL_Free(s);
    }
    break;
    case 0x0402: //RunningData 返回
        printf("Runningdata %s\r\n", ErrMsg);
        printf("data: %s\r\n", TSL_ObjToStr(L, ResultObj));
        printf("\r\n");
        break;
    }
}

void finalcallback(TSL_State* L, intChannelId)
{
    printf("End ChannelId:%d\r\n", ChannelId);
}

    constchar* s, *s1;
    LWrap* lwrap = TSL_InterpNewLWrap();
    TSL_State* L = TSL_InterpGetLFromWrap(lwrap);
    if(!TS_GetLogined())//没有登录,退出
    {
        printf("login failed\r\n");
        TSL_InterpFreeLWrap(lwrap);
        return ;
    }
    s = "echo 'Hello1'; for i:=0 to 1 do begin
sendrunningdata(rand(10,1),'Hello1'); sleep(1000); end;return
array(1,2,3);";
    TObject* r = TSL_NewObject();
    int ChannelId = TS_SendExecuteAndWaitEx(L, s,
(void*)TSL_GetGlobalNilObj(), r, NULL, NULL, true, callback, finalcallback);
    Sleep(2000);
    printf("\r\nChannelId:%d\r\n", ChannelId);
    TSL_FreeObj(L, r);
    TS_ProcessMessages(); //如果是 GUI 程序，就直接由主消息循环来处理消息，异步是一
个线程内的，也就是说回调的线程就是发起调用的线程
    TSL_InterpFreeLWrap(lwrap);
```

5.4 Tsl 语言插件

TSL 插件可以实现对 TSL 语言的扩展，通过 C/C++等新增函数或类。比如用户发现 TSL 现有的函数中缺少需要的函数，就可以通过 tslL_register 注册的方式新增。如果是新增类，则需要通过 TSL_NewAnyMan 和 TSL_RegisterAnyMan 函数来新增。

5.4.1 新增函数

新增函数使用 tslL_register 方式注册

```
void tslL_register(TSL_State* L, constchar* name, TSL_CFunction func);
```

说明：用户通过该函数注册，可以将一个C或C++的函数，编译注册为天软函数，在天软中直接调用。

参数：

L:解释器状态指针

name:函数名，该名称必需为小写字符串。

func:插件函数指针

```
typedefint(*TPKG_Init)(TSL_State* L, OpenLibProc proc);
```

说明：作为TSL语言的插件，DLL应放入plugin目录，且应导出名为PKG_Init的函数，在PKG_Init可以调用proc来注册一组二进制函数,也可以在该函数中使用tslL_register一个个注册函数

```
void tslp_error(TSL_State* L, constchar* s);
```

说明：在二进制函数中，设置出错信息并抛出解释器可捕获异常中止函数的执行

参数：

L:解释器状态指针

s:错误信息。

范例:

这是 c++的代码,新建一个 dll 工程,将输出目录设置到为天软安装目录的子目录 Plugin 目录,然后启动天软客户端采用本地运行模式运行或者用天软安装目录下的 tsl.exe 执行下面的代码

```
c := test_extra_func(5,50,v);
echo "c:",c,"\r\n";
echo "v:",v,"\r\n";

extern"C" {
    #include"TSSVRAPI.h"
    __declspec(dllexport) int  PKG_Init(TSL_State* L, OpenLibProc proc);
}
int test_extra_func(TSL_State* L, TObject* iArgs, intiNum, TObject* oResult)
{
    if (iNum != 3)
        returnTSL_ERROR;//参数判断
    TObject* paramA = iArgs;
    if(!TSL_NumberCheck(paramA) || !TSL_NumberCheck(paramB))//参数判断
    {
        tslp_error(L, "param error:it needs number!\r\n");
        returnTSL_ERROR;
    }
```

```
    TObject* paramB = TSL_ReadTObjectFromPtr(iArgs, 1);//获取参数
    TObject* outMulti=TSL_GetParamRef(L, 2);//第三个参数
    TSL_SetReal(L, oResult, TSL_AsDouble(paramA) + TSL_AsDouble(paramB));//
设置返回值
    TSL_SetReal(L, outMulti, TSL_AsDouble(paramA) * TSL_AsDouble(paramB));
    returnTSL_OK;
}
int PKG_Init(TSL_State* L, OpenLibProcproc)
{
    tslL_register(L, "test_extra_func", test_extra_func);//注册函数
    returnTSL_OK;
}
```

5.4.2 新增类

为实现新增类，必须实现类的构造函数、析构函数、类成员属性的存取和类方法的调用。

天软为用户提供了下列函数：

1）创建 TSL 类

```
TAnyMan* TSL_NewAnyMan(char* classname, TSL_NewAnyP newp, TSL_FreeAnyP freep, TSL_MethodP
methodp, void* reserved, void* reserved2);
```

说明：创建一个TSL类管理器

classname:类名，应小写。此classname应为永久串不能被释放

newp:创建一个实例的回调函数。

freep:释放对象时的回调函数。

methodp:调用对象的方法或者属性时的回调函数

reserved:保留含义，必需置为NULL

reserved2:保留含义，必需置为NULL

返回:一个类管理器指针

2）释放一个 TSL 类管理器

```
void TSL_FreeAnyMan(TAnyMan* anyman);
```

3）注册一个 TSL 类管理器

```
void TSL_RegisterAnyMan(char* t, TAnyMan* anyman);
```

参数：

t：类名，应小写。

anyman:注册的类管理器。

4）查找一个 TSL 类管理器

```
TAnyMan* TSL_FindAnyMan(char* t);
```

参数：

t：类名，应小写

返回：类管理器

5）创建一个二进制 TSL 对象的存贮指针

```
TAny* TSL_NewAny(TSL_State* L, char* t, TObject* iArgs, int iNum);
```

参数：

L:解释器状态指针

t:类名

iArgs:创建的构造函数参数数组指针

iNum:创建的构造的参数个数

返回:对象存贮指针

6）释放一个 TSL 对象存贮指针

```
TAny* TSL_FreeAny(TSL_State* L, TAny* any);
```

参数

L:解释器状态指针

any:一个TSL对象的存贮指针

返回：如果释放后对象引用计数为 0，则返回 NULL,否则返回 any

7）自动加 1

```
void TSL_AnyAddRef(TAny* any);
```

说明：给一个对象的引用计数加 1，当复制一个 TObject 的时候，引用计数会自动加 1，但一旦需要自己存贮 any 对象（例如创建一个面向对象的聚合的时候），可能需要使用到主动加引用计数防止其他代码释放。使用完毕后用 TSL_FreeAny 来减去引用计数

8）获得 AnyMan 的信息

```
void TSL_GetAnyMan(TAnyMan* anyman, char** classname, TSL_NewAnyP* newp, TSL_FreeAnyP* freep, TSL_MethodP* methodp, void** reserved, void** reserved2);
```

9）检查是否一个 TSL 对象类型

```
bool TSL_AnyCheck(const TObject* p0);
```

10）从 TSL 对象类型中获得存贮指针

```
TAny* TSL_GetAny(TObject* v);
```

11）从一个对象的存贮指针中获得对象自己建构的数据指针

```
void* TSL_GetDataFromAny(TAny* any);
```

12）设置一个对象的存贮指针中的数据指针

```
void TSL_SetDataToAny(TAny* any, void* data);
```

13）获得一个对象存贮的 TSL 类管理器

```
TAnyMan* TSL_GetAnyManFromAny(TAny* any);
```

14）设置一个 TSL 数据为 TSL 的面向对象类型

```
void TSL_SetTypeAny(TSL_State* L, TObject* obj, TAny* any);
```

参数：

L:解释器状态指针

obj:TSL对象

any:一个TSL面向对象的存贮指针

新增类的范例请参考天软 API 接口中 tslsamplugin 范例。